

Q-Rex User Manual

1. Installation

Option A — standalone desktop application (no installation of any runtime).

Copy the folder `release/q-rex-<platform>-<arch>/` to the target machine and run `q-rex` (`q-rex.exe` on Windows). No internet connection or Node.js is needed. Build for another platform on a machine with Node ≥ 20 : `npm install && node scripts/build-exe.mjs win32 x64` (or `darwin`, `linux`).

Option B — single file. Open `dist-single/q-rex.html` in Chrome, Edge or Firefox. It is fully self-contained and works offline.

Option C — from source. `npm install`, then `npm run dev` (GUI at <http://localhost:5173>), `npm test` (38 tests), `npm run bench -- 3 20` (headless scenario suite), `npm run e2e` (browser Benchmark-2 check; needs ffmpeg and chromium).

2. Quick start

1. Start the application. The simulation runs immediately: the virtual camera starts at the screen centre, finds the beacon on the wide view, slews to it and locks (state turns green `TRACK`, typically within 1 s).
2. Open the sections in the left panel to change parameters. Disturbance and tracker controls act **live**; scene controls restart the run.
3. Watch the status tiles. A tile turns green when its spec target is met and red when it is missed.
4. At the end of a timed run the performance report (JSON), an HTML report and the per-frame CSV log are saved automatically (browser download folder). Buttons `JSON / CSV LOG / HTML export` at any time.

3. GUI description

Left panel

- *Run / Restart, Sim speed ($\frac{1}{4}\times \dots \text{max}$), Show ground truth overlay, Auto-save report at end of run.*

- *Load scenario*: applies a named benchmark scenario (A nominal, B single disturbance at spec maximum, C all maxima, plus dropouts and decoys).
- Parameter sections (all values clamped to the specification ranges):

Section	Parameters
Scenario & Run	random seed, run duration (0 = endless)
Virtual Camera	camera type (monochrome default / colour), screen size (≥ 2000), camera width/height, FOV (default 4° horizontal), update rate (≥ 30 Hz), max pan/tilt speed 5–10 °/s, gimbal acceleration, extra command latency
Target	motion (straight, circular, figure-8, random, spiral, sinusoidal, user waypoints), speed, size 5–20 px, shape, number of targets (1 beacon + decoys), initial location (random / user-defined), waypoints (x,y per line)
Image Noise	salt & pepper (density ≤ 10 %), Gaussian ($\sigma \leq 20$), Poisson — any combination
Jitter, Platform, Atmosphere	jitter ($\pm$ px/frame ≤ 20), platform motion (linear / circular / random / spiral / figure-8, ≤ 20 px/frame), atmosphere (clear, haze, fog, rain, low light) with severity, beacon dropouts, turbulence
Tracker	acquisition mode (wide-view overview / raster scan), control gain, detection threshold, coast frames

- A yellow warning appears when target speed plus platform speed exceeds the gimbal limit (saturation expected).

Status bar — current state (SEARCH, SLEW, TRACK, COAST, REACQUIRE) and tiles: acquisition time (≤ 2 s), mean tracking error with RMSE/max (≤ 10 px), centroid error RMSE/P95, target loss and lock retention (< 5 %), maximum re-acquisition time (≤ 1 s), tracker FPS and ms/frame (≥ 20 FPS).

Virtual camera view — the 640×480 image with overlays: red cross = boresight, green square = detection, yellow circle = filter estimate, cyan dashed box = search window, magenta circle = ground truth (optional). **Screen map** — the whole 2000×2000 screen with the camera footprint (blue box), target trail, true beacon (magenta) and estimate (green). **Live error** — boresight error of the last 300 frames with the 10 px line and a colour strip of tracker state.

4. Benchmark 1 — scenario suite

Set seconds per run and number of seeds, press *Run all scenarios*. Each scenario runs headlessly; the table shows acquisition, error, centroid RMSE, loss, lock, re-acquisition, FPS and pass counts (green = all runs met the spec check). *Export suite JSON* saves the

full metrics of every run. The same suite runs without the GUI: `npm run bench -- <seeds> <seconds> [filter]`.

5. Benchmark 2 — video input

1. In the *Benchmark 2* panel choose the `.mp4` (full screen of any size, or a 640×480 camera image); optionally choose a ground-truth CSV with lines `frame,x,y` (0-based frame index, pixel-index coordinates). Set the video frame rate (30).
2. Press *Run on video*. The PTZ model is bypassed: every frame is decoded exactly (seek per frame), converted to grey, and passed to the same detector/tracker logic. The preview shows the detection.
3. The result table gives frames, processing FPS, acquisition time, lock retention, re-acquisition events/time, and — with ground truth — centroiding error mean/RMSE/P95/max. *Centroid log CSV* contains `frame,t,found,x,y,state,procMs` per frame; coordinates use the OpenCV pixel-index convention (centre of the first pixel = 0.0).

Frames ≥ 1000 px are treated as the whole screen: search runs on a 4× binned copy, tracking on a window at full resolution.

6. Performance report fields

`simulationDurationSec`, `averageFPS`, `endToEndFPS`, `processingTimeMeanMs/P95/Max`, `acquisitionTimeSec`, `timeToCentreSec`, `trackingErrorMeanPx/RmsePx/P95/P99/MaxPx`, `trackingErrorInclJitterMeanPx`, `centroidingError...`, `lockRetentionRatePct`, `targetLossPct`, `reacquisition...`, `gimbalSaturationPct`, per-requirement pass/fail checks, and the complete configuration used (so a run can be reproduced from the seed).

7. Troubleshooting

- *Acquisition > 2 s*: the *Raster scan* mode cannot meet 2 s by physics (12 s sweep); use *Wide-view overview*.
- *Red tracking-error tile with platform + jitter at maximum*: expected limitation (see technical report §9).
- *Video fails to load*: the browser must be able to decode the codec (H.264 is supported by Chrome/Edge/Electron).
- *Slow GUI*: set Sim speed to 1×, or use the headless `npm run bench`.